Programowanie w systemie Linux

Narzędzia programistyczne

Systemy operacyjne - laboratorium

Informatyka – S1, rok I, semestr 2

Prowadzący: dr hab. inż. Paweł Ziemba, prof. PM i US

GCC

* GCC (ang. GNU Compiler Collection) – zestaw kompilatorów GNU dla wielu języków programowania i systemu Linux (dla Windows też).
* gcc – kompilator C,
* g++ - kompilator C++.

Polecenie gcc i g++

* *Użycie:*

  $ gcc [opcje] [plik_źródłowy]

  $ g++ [opcje] [plik_źródłowy]

* Wynik kompilacji zapisywany jest domyślnie w pliku *a.out*
* Plik źródłowy powinien posiadać rozszerzenie *.c (dla gcc) lub *.cpp (dla g++).
* Plik wynikowy nie powinien posiadać rozszerzenia.
* Opcje podawane w formie **–opcja.**

Podstawowa różnica między kodami C i C++

* Za pomocą narzędzia gcc zaleca się kompilację kodów języka C.
* Kody języka C++ powinny być kompilowane narzędziem g++.
* Przy podawaniu plików nagłówkowych w kodach C++ unika się używania rozszerzeń *.h (nowsze pliki nagłówkowe), np.:
  * zamiast: *#include <iostream.h>*
  * będzie: *#include <iostream>*
  * zamiast: *#include <math.h>*
  * będzie: *#include <cmath>*

Najważniejsze opcje poleceń gcc i g++

* *Użycie:*

  $ gcc [opcje] [plik_źródłowy]

  $ g++ [opcje] [plik_źródłowy]

* Opcje:
  * -o [nazwa]-> kompilacja do pliku wynikowego o wskazanej nazwie,
  * -c -> kompilacja bez linkowania bibliotek, na wyjściu otrzymywany jest plik *.o,
  * -S -> kompilacja i assemblacja, bez linkowania, na wyjściu otrzymywany jest kod assemblera (*.s),
  * -x [język] – określenie języka plików źródłowych, możliwe języki (wartości): c, objective-c, c-header, c++, cpp-output, assembler, assembler-with-cpp .

Najważniejsze opcje poleceń gcc i g++

* *Użycie:*

  $ gcc [opcje] [plik_źródłowy]

  $ g++ [opcje] [plik_źródłowy]

* Opcje:
  * -ansi -> sprawdzenie zgodności kodu ze standardem ANSI C,
  * -Wall -> wypisuje ostrzeżenia dla wszystkich niezalecanych konstrukcji języka,
  * -p -> opcja „pedantic” – wyświetla wszelkie niezgodności ze standardem ANSI C,
  * -l[biblioteka] -> (bez spacji) – dołączenie bibliotek (np. –lm dołącza bibliotekę libm.a),
  * -g -> dodatkowe informacje dla debugera, opcja pozwala na późniejsze debugowanie programu.

Program make

* Użycie:

  $ make [opcje]

* Zarządca kompilacji, korzystający z pliku makefile lub Makefile.
* Potrafi „dostrzec”, które pliki źródłowe programu uległy zmianie i kompiluje tylko te pliki (oszczędność czasu przy rozbudowanych projektach).
* Zdejmuje z programisty ciężar pisania kolejnych poleceń gcc (g++) w lini komend przy kompilacji rozbudowanych projektów.

Opcje programu make

* Użycie:

  $ make [opcje]

* Opcje:
  * -k -> make kontynuuje działanie gdy znajdzie błąd w kodzie,
  * -n -> make wypisze „co by robił”, ale w rzeczywistości nie kompiluje kodu,
  * -f [nazwa] -> podanie programowi make innego pliku niż domyślny o nazwie makefile lub Makefile,
  * -s -> make nie wypisuje na ekranie wykonywanych poleceń.

Przydatność programu make

* Założenia:
  * Program składa się z trzech plików źródłowych (main.c, 2.c, 3.c) i trzech plików nagłówkowych (a.h, b.h, c.h).
  * Plik main.c korzysta z nagłówka a.h i funkcji w plikach 2.c i 3.c.
  * Plik 2.c korzysta z nagłówków a.h i b.h.
  * Plik 3.c korzysta z nagłówków b.h i c.h.
* Jeżeli programista zmieni plik c.h to musi pamiętać, żeby ponownie skompilować plik 3.c.
* Aby o tym nie zapominać może zawsze wykorzystywać komendę:

  $ gcc -o program main.c 2.c 3.c

  ale wtedy niepotrzebnie kompiluje pliki niezmieniane i nie korzystające ze zmienionych nagłówków (strata czasu BARDZO odczuwalna przy projektach złożonych z dużej ilości plików źródłowych).

Przydatność programu make - plik makefile

* Założenia:
  * Program składa się z trzech plików źródłowych (main.c, 2.c, 3.c) i trzech plików nagłówkowych (a.h, b.h, c.h).
  * Plik main.c korzysta z nagłówka a.h i funkcji w plikach 2.c i 3.c.
  * Plik 2.c korzysta z nagłówków a.h i b.h.
  * Plik 3.c korzysta z nagłówków b.h i c.h.
* Plik makefile dla przedstawionych zależności:

```
moj_program: main.o 2.o 3.o
    gcc –o moj_program main.o 2.o 3.o
main.o: main.c a.h
    gcc –c main.c
2.o: 2.c a.h b.h
    gcc –c 2.c
3.o: 3.c b.h c.h
    gcc –c 3.c
```

Dziękuję za uwagę